



One Prism ID. One home across chains.

A persistent identity, financial coordination, and private relationship protocol for the networks where people actually live.



STATUS

Proposed product and concept synthesis

EVIDENCE CEILING

Core v1: X2 local implementation

Prism / persistent home, native venues

VERSION

0.1.0

REPOSITORY

etvjay/Prism

2026-09-01

Contents

This document defines the product thesis, architecture, user experience, requirements, evidence boundaries, and roadmap for Prism. It is intentionally explicit about what is implemented, proposed, deferred, and not yet proven live.

1. Executive definition

2. Product status and evidence boundaries

3. Product thesis

4. Problem statement

5. Primary users and jobs

6. Product principles and non-negotiable invariants

7. Full concept model

8. System architecture

9. Product surfaces and information architecture

10. Functional requirements

11. Core workflows

12. State models

13. Data model requirements

14. API and SDK requirements

15. Privacy and threat requirements

16. Product quality requirements

17. Evidence and release requirements

18. Release roadmap

19. Scope boundaries

20. Success metrics

21. Risks and mitigations

22. Open decisions

23. Traceability map

24. Definition of done for the full Prism concept

Sources

1. Executive definition

Prism is a Starknet-native identity, financial coordination, and private relationship protocol that gives a person one persistent Prism ID across chain-specific accounts, social identities, private financial state, and communication relationships.

The user experiences one coherent home. Underneath that home, each network remains native, each authority remains explicit, and each privacy claim is limited to what the underlying mechanism can actually prove.

Product sentence

Prism gives a person one persistent ID and one home for accounts, money, social identities, private payments, relationships, and communication across the networks where those things actually live.

Technical sentence

Prism separates persistent identity from venue-specific execution and external social identity, then connects them through verifiable, revocable, purpose-specific authority and resolution.

Short sentence

One Prism ID. One home across chains.

Governing principle

Prism is the persistent home. Networks are execution venues. Social apps are identity surfaces. STRK20 is private financial infrastructure. Authority and resolution connect them.

2. Product status and evidence boundaries

Prism has one product direction and several release or evidence tracks. A release track is not a separate product or repository.

Track	Purpose	Current state	Does not yet prove
Core v1	Identity, authority, binding, resolution, revocation, governed operations, durable backend state, and truthful Home surfaces	Local implementation and gates at X2	Live deployment, testnet receipts, independent chain reads, or mainnet readiness

Track	Purpose	Current state	Does not yet prove
STRK20	Private Starknet financial state and private action surface	Local Wallet API, state, receipt, and adapter implementation at X2	A supported live wallet, private pool action, accepted receipt, private-state readback, or mainnet eligibility
Social principals	Human-addressed identity and payment resolution	Product and system model specified; provider work remains staged	A production social OAuth integration or public identity graph
PrismClaim	Private payment to an unregistered recipient followed by onboarding and claim	Product and contract concept; implementation is a separate decision	A production contract or STRK20-compatible pool route
PrismChannel	Private relationship channel carrying payment memos, receipts, claims, and authorization requests	Minimal testnet concept and data model	A full messenger or production encrypted communication network
Continuity	Rotation, recovery, delegation, guardians, successors, and inheritance	Long-term product direction	MVP or sprint capability

Current evidence vocabulary

- **X0:** hypothesis, proposal, or unobserved live claim.
- **X1:** fixture or mock behavior.
- **X2:** local controlled implementation with reproducible tests.
- **X3:** observed testnet operation with receipts and independent reads.
- **X4:** repeated or independently reproduced operation.
- **X5:** independently verifiable mainnet or production evidence.

Local tests, simulated wallets, injected providers, dry-runs, local Postgres, and mock protocol calls remain X2 evidence. A transaction hash is not enough to prove a product outcome or privacy property.

3. Product thesis

People currently manage identity and authority as a collection of unrelated infrastructure identifiers:

```

Starknet address
Base address
another wallet
Telegram handle
X handle
email address
phone number
chat thread
payment link

```

Those identifiers are not interchangeable. A wallet address is an execution identity. A social username is presentation and discovery metadata. An email address is an external principal. A private balance belongs to a privacy system. None of them should silently become the user's durable identity root.

Prism introduces a persistent identity object above those systems:

```
Person
-> PrismIdentity
    -> controllers and authorities
    -> native execution identities
    -> verified external principals
    -> purpose-specific resolution
    -> public and private financial capabilities
    -> private relationships and communication
```

The product does not hide the existence of different networks. It removes the requirement that the user manually reason about every network, address, key, and protocol before accomplishing an ordinary human task.

The central insight

A person should be able to say:

```
Pay Alice
```

rather than:

```
Find Alice's current address on the correct chain,
choose the correct privacy route,
select the correct account,
verify the recipient,
and hope the resulting transaction is attributed correctly.
```

Prism turns human identity and persistent authority into a resolution problem:

```
identity + purpose + venue + privacy policy
-> authorized capability
```

4. Problem statement

User problem

A person with assets and relationships across multiple networks cannot reliably answer:

- Which accounts currently represent me?
- Which account should execute this action?
- Which social identity belongs to that person?
- Can I pay someone without knowing their wallet address?
- What remains true if I rotate or revoke an account?
- Which part of this action is private?
- What authority did I grant?
- Did the action merely submit, or did it complete and reconcile?

System problem

Existing products often collapse distinct concepts:

```
identity = address
controller = execution account
social handle = stable user identity
submitted = completed
receipt = product outcome
private route = all-chain privacy
backend cache = canonical state
```

Those collapses create security, privacy, recovery, and comprehension failures.

Product opportunity

Prism can become a durable coordination layer for people and applications by keeping the distinctions intact while presenting a unified experience.

The opportunity is not to create another wallet dashboard, address book, messenger, bridge, solver, or social network. Those are possible surfaces. The deeper product is persistent, user-controlled digital authority that can be addressed through human identities and resolved into the correct native execution or private relationship capability.

5. Primary users and jobs

5.1 Primary user: multi-venue asset owner

A person who controls accounts on Starknet and Base and wants one durable identity across them.

Jobs to be done:

1. Create one persistent Prism identity.
2. Connect a native execution account without replacing the Prism identity.
3. Prove control of an external account.
4. Bind, resolve, rotate, or revoke an account safely.
5. See public and private financial state with accurate provenance.
6. Send or receive value using a human identity rather than a raw address.
7. Understand what is private, what is public, and what is pending.

5.2 Privacy-conscious payer and recipient

A person who wants private note-to-note transfers through a supported STRK20 wallet route.

Jobs to be done:

1. Determine whether the connected wallet supports the required STRK20 capability without triggering unnecessary private-data consent.
2. Shield assets through a clear two-step flow.
3. Wait for note maturity without being shown a false ready state.
4. Consent intentionally to a private-balance read.
5. Execute a private transfer and receive an honest receipt.

6. Understand public deposit metadata, private transfer fields, and remaining correlation risks.

STRK20 supports shielded balances, private transfers, and private application execution. Registration, deposit metadata, and some application-side amounts or timing remain visible depending on the route.[10][11]

5.3 Human-addressed recipient

A person who expects to be found through a stable external identity such as Telegram or X rather than a wallet address.

Jobs to be done:

1. Bind an external principal to a Prism ID.
2. Change a display handle without losing the underlying binding.
3. Receive a payment request without exposing unnecessary wallet details.
4. Onboard later if a sender created a claim before the recipient had Prism.

5.4 Application or agent integrator

A developer or authorized agent that wants to create intents, resolve identities, request approvals, inspect receipts, or send relationship-scoped actions without receiving private keys or viewing keys.

Jobs to be done:

1. Use stable domain-first API and SDK vocabulary.
2. Read canonical or derived state with provenance.
3. Create an intent without bypassing policy.
4. Request a pause, verification, escalation, or approval.
5. Poll an operation through its full lifecycle.
6. Never sign, release, or mark settlement complete without the required authority.

5.5 Auditor, operator, or evaluator

A skeptical reviewer who needs to understand what is canonical, what is derived, what happened, and what remains unverified.

Jobs to be done:

1. Trace product claims to implementation and evidence.
2. Verify network, block, transaction, operation, or request identifiers.
3. Distinguish local, testnet, mainnet, and public evaluator evidence.
4. Confirm that failures and privacy limitations are legible.

6. Product principles and non-negotiable invariants

6.1 Identity is persistent

A Prism ID is a durable protocol identity independent of any one execution address, social username, device, or chain.

```
PrismID != Starknet address
PrismID != Base address
PrismID != Telegram username
PrismID != X username
```

6.2 Authority is explicit

The identity, the controller, the execution account, the authenticated product session, and the private wallet authority are separate concepts.

```
identity != controller != execution identity
```

6.3 Execution remains native

A Base action requires Base-valid authority. A Starknet action requires Starknet-valid authority. Prism coordinates and resolves; it does not pretend that one signature authorizes every venue.

6.4 Resolution is purpose-specific

The resolver must not return a generic address when the action requires a venue, purpose, privacy policy, or capability decision.

```
resolve(identifier, purpose, venue?, privacy?)
-> authorized capability or explicit failure
```

6.5 Revocation is real

A revoked binding must never be returned as active. Revoking one binding must not destroy the parent Prism ID.

6.6 Resolvable does not mean publicly enumerable

A social identity or private relationship may be resolvable for an authorized interaction without exposing a downloadable public graph of social accounts, Prism IDs, and wallets.

6.7 Privacy is scoped to the mechanism

No product surface may claim that all Prism activity is private. Privacy claims must name the observer, hidden datum, visible datum, and relevant correlation assumptions.

6.8 Consent is explicit

Private-balance reads and other private-state access require intentional user consent. Capability detection must not call a private-balance method as a side effect.

6.9 Submitted is not completed

Every chain-dependent operation must distinguish:


```
created
-> awaiting_authorization
-> ready
-> submitted
-> processing
-> confirming
-> confirmed
-> indexed
-> reconciled
-> completed
```

Timeout, pending, reverted, failed, expired, cancelled, and requires-attention states remain distinct.

6.10 Derived data is not authority

The backend, indexer, cache, Activity feed, and portfolio aggregate may be useful representations. They do not become canonical merely because Prism displays them.

6.11 Keys remain compartmentalized

```
Starknet execution key
!= Base execution key
!= STRK20 viewing key
!= communication key
!= product authentication credential
```

Prism application code must not receive or persist a user's STRK20 viewing key in the normal Wallet API route.

6.12 Evidence is part of the product

Every consequential claim needs a reproducible evidence record. A local implementation, an attractive UI, a successful submission response, or a transaction hash alone does not prove the entire product claim.

7. Full concept model

7.1 Prism Home

Home is not merely a wallet dashboard. It is the persistent place where Prism represents:

```
Prism ID
-> identity
-> controllers and authorities
-> native accounts
-> public balances and positions
-> private balance capability
-> social principals
-> private payments
-> claims
-> channels and relationships
-> activity and receipts
```

The first two seconds of Home should communicate:

This is my financial and digital home.

The architecture may become visible through detail views, but the primary surface should be understandable without an architecture lecture.

7.2 Prism identity

A `PrismIdentity` is a persistent object rooted in Starknet. It remains stable while controllers, execution identities, social handles, devices, or venue accounts change.

Minimum identity capabilities:

- create;
- read;
- display a stable human-facing identifier;
- associate a controller;
- expose active and revoked binding state through authorized resolution;
- preserve identity continuity through binding changes.

7.3 Controller

The controller is the authority currently allowed to mutate protected Prism state. Controller authority is not the same as product login or an external social identity.

The Starknet registry is canonical for controller state, accepted bindings, binding revocation, and resolution-critical identity state.

7.4 Execution identity

An execution identity is a venue-native account that performs actions on a specific network. Prism may resolve to it for a purpose, but the account remains native to its venue.

Example:

```
Prism ID      prism:P7F21
Controller    Starknet account A
Base account  0x83A...
```

7.5 External principal

An external principal is a verified identity from a social or identity platform.

```
ExternalPrincipal {
  platform
  subject_id
  current_handle
  prism_id
  verification
  visibility
  verified_at
  revoked_at
}
```

The stable platform subject is the identity key where the platform exposes one. The current username is presentation metadata and must not silently transfer authority when reassigned.

Initial provider priority:

- 1. Telegram;
- 2. X;
- 3. email, phone, Discord, GitHub, Farcaster, and other providers as separate integrations.

7.6 Resolution

Resolution is the core coordination primitive:

```
human identifier or Prism ID
+ purpose
+ venue
+ privacy policy
-> permitted capability
```

Examples:

```
resolve(prism:P7F21, BASE_EXECUTION)
-> active Base execution identity

resolve(@alice, TELEGRAM, PRIVATE_PAYMENT)
-> verified principal
-> Prism ID
-> private receive capability

resolve(@alice_xyz, X, COMMUNICATION)
-> verified principal
-> Prism ID
-> PrismChannel capability
```

Resolution must fail closed when the principal is unverified, revoked, ambiguous, stale beyond policy, or incompatible with the requested purpose.

7.7 Financial coordination

Prism coordinates public and private financial actions without pretending that all balances are stored in a single universal ledger.

Canonical sources remain venue-specific:

Financial fact	Canonical source
Public Starknet balance	Starknet
Public Base balance	Base
STRK20 private state	STRK20 pool through a supported wallet route
Portfolio aggregate	Derived Prism representation
Operation state	Prism operation store plus verified execution evidence
Receipt	Reconciled provider and domain evidence

7.8 STRK20 private financial surface

STRK20 is Prism's first private financial and private Starknet execution surface. The default consumer route is:

```
Prism dapp
-> get-starknet / wallet discovery
-> starknet.js / WalletAccountV6
-> Privacy Wallet API
-> privacy-enabled wallet
-> STRK20 pool
```

The wallet owns registration, viewing keys, notes, proving, and supported private operations. The underlying Starknet Privacy stack separates wallet, SDK, discovery, proving, contract, and anonymizer concerns; Prism owns product intent, capability detection, identity resolution, operation UX, receipts, privacy labels, and any Prism-owned application contracts.[12]

The product must communicate the following lifecycle clearly:

```
capability unknown
-> wallet/network verified
-> registration required
-> approval pending
-> shielding
-> confirmed
-> maturing
-> privately available
-> proving
-> transfer pending
-> transfer confirmed
```

A normal shield includes an ERC-20 approval and a pool deposit. Fresh notes generally require roughly ten blocks before later spending. Pool fees must be read from the current pool rather than hard-coded. Private transaction senders may be rotating relayers, so sender address is not a reliable user-activity identity field.[10] [11]

7.9 PrismClaim

PrismClaim addresses the case where a sender knows a person through a verified human identifier but the recipient has not onboarded to Prism.

Conceptual object:

```
PrismClaim {
  claim_id
  external_principal_commitment
  asset
  amount
  expiry
  sender_policy
  status
}
```

Flow:

```
sender enters @alice
-> Alice has no Prism ID
-> Prism verifies the external principal
-> sender creates a private claim
-> Alice receives an invitation
-> Alice onboards and proves control
-> Alice creates or links a Prism ID
-> Alice claims the funds
```

Expiry flow:

```
claim expires
-> refund becomes available to sender
-> refund executes
-> claim closes
```

PrismClaim is a candidate for the first Prism-owned STRK20 application action because it expresses the full product idea: human identity, private value, onboarding, persistent identity, and claim settlement. It remains a proposed mechanism until its contract interface, pool integration, privacy properties, and live evidence are accepted.

7.10 PrismChannel

PrismChannel is a private relationship capability between Prism identities, not a general chat application.

A channel may carry:

- encrypted messages;
- payment memos;
- payment requests;
- claim invitations;
- invoices;
- settlement proposals;
- authorization requests;
- receipts;
- future delegation or recovery requests.

Conceptual channel:

```
PrismIdentity A
-> PrismChannel
    -> PrismIdentity B
```

Communication keys are separate from Starknet account keys, Base account keys, and STRK20 viewing keys. Plaintext message content, social handles, payment memos, and relationship metadata must not be placed on a public chain by default.

Initial channel scope is one relationship, one encrypted payment memo or receipt reference, authorized participant read/decrypt, and revocation or archive. It is not a full messenger, public social graph, automatic wallet signer, or payment authority.

7.11 Continuity

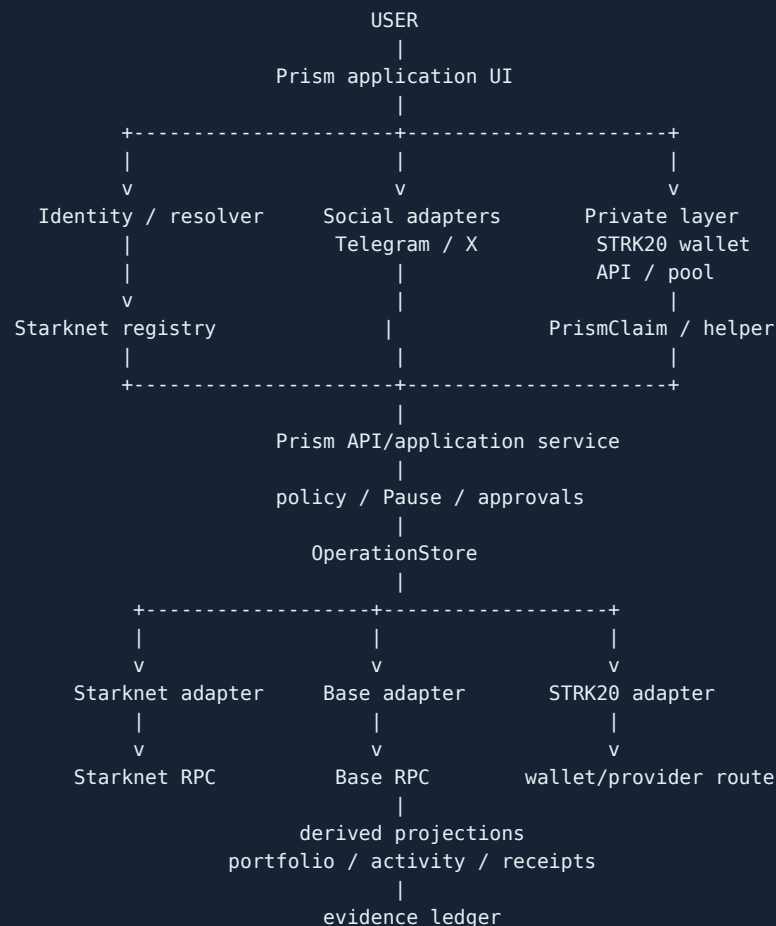
Prism is a continuity protocol in the long-term product direction. A Prism ID may survive:

- wallet or account rotation;
- chain migration;
- social username changes;
- device replacement;
- loss of an execution account;
- incapacity or death.

Future capabilities include guardians, successors, recovery, delayed transfer, beneficiary policies, delegated agents, and bounded authorities. These capabilities must remain architecturally possible without becoming MVP dependencies.

8. System architecture

8.1 Logical architecture



Communication is a related but separate path:

```

PrismIdentity
-> PrismChannel
  -> encrypted payload or content reference
  -> channel index and authorized discovery
  
```

8.2 Canonical authority map

Domain object or state	Canonical authority	Prism responsibility
Prism identity	Starknet Prism registry	Present, resolve, and reconcile
Controller state	Starknet Prism registry	Enforce caller and lifecycle rules
Accepted venue binding	Starknet registry after controller-authorized transition	Prepare proof, submit operation, reconcile
Base ownership proof validity	Trusted backend verifier under accepted proof ladder	Verify EOA, EIP-1271, and ERC-6492 forms
Public Base balance	Base	Read and label provenance
Public Starknet balance	Starknet	Read and label provenance
STRK20 private balance	STRK20 pool through supported wallet	Request intentional consent and display provider result
Social subject identity	External provider authentication	Store minimum verified association
Current social handle	External platform	Display as mutable presentation metadata
Portfolio aggregate	Derived Prism projection	Mark as derived and timestamped
Operation state	Prism operation model plus provider evidence	Preserve lifecycle and recovery semantics
Communication ciphertext	Channel storage layer	Enforce key separation and access policy
Sprint transaction eligibility	Upstream STRK20 validator	Reproduce checks before ledger inclusion

Derived data may be displayed, cached, or aggregated. It may not silently override canonical state.

8.3 Contract boundaries

PrismIdentityRegistry

Owns:

- identity creation;
- controller state;
- binding state;
- revocations;
- resolution-critical state;
- replay protection for accepted proof digests;
- canonical events.

PrismClaim

Potentially owns:

- claim creation;
- external-principal commitment;
- asset and amount terms;
- expiry;
- claim authorization;

- refund lifecycle;
- pool-integrated settlement.

PrismChannelRegistry or helper

Only if justified by accepted product and system decisions. It may own:

- channel commitment;
- encrypted payload or content-addressed pointer commitment;
- channel status;
- participant or key commitment references.

No contract should exist only to make an architecture diagram look complete. Contract boundaries must follow authority, state, privacy, and evidence requirements.

8.4 Backend module boundaries

The backend is a modular monolith until scale or isolation requirements justify decomposition.

```
identity/  
controllers/  
bindings/  
resolver/  
principals/  
social/  
claims/  
channels/  
portfolio/  
privacy/  
operations/  
receipts/  
evidence/  
policy/
```

Adapters:

```
starknet/  
base/  
strk20/  
telegram/  
x/  
storage/  
indexer/
```

The backend prepares, verifies external facts, derives projections, persists operation state, and coordinates execution. It does not become the canonical owner of Starknet identity state.

8.5 One authority path

REST, SDK, and MCP must converge on one application and policy path:


```

REST client  -+
SDK client   -+--> Prism API/application service
MCP client   -+
              |
              v
            Prism Pause
              |
              v
          OperationStore
              |
              v
    execution adapter / chain

```

No SDK or MCP method may bypass identity authority, Base proof verification, Pause policy, approval requirements, operation lifecycle, or reconciliation.

9. Product surfaces and information architecture

9.1 Primary navigation

```

Home
Activity
Connections
Profile

```

Send and Receive are primary actions, not necessarily permanent navigation destinations. A lightweight relationship entry point may appear inside Home or Activity after PrismChannel is proven useful.

9.2 Home

Home hierarchy:

```

identity
-> total financial state
-> private financial state
-> primary actions
-> accounts and people
-> activity
-> evidence and provenance

```

Home must show:

- Prism ID;
- total state with provenance and timestamp;
- private state only after intentional wallet consent;
- Send and Receive actions;
- Starknet and Base account status;
- verified social principals;
- recent activity;
- pending and failed operation states;

- privacy labels;
- technical detail on demand.

Home must not show fabricated balances, fabricated activity, hidden mock transactions, or a private badge that implies more privacy than the underlying route provides.

9.3 Connections

Connections groups:

```
Accounts
  Starknet
  Base

Social principals
  Telegram
  X

Private capability
  Wallet support
  Network
  consent status

Future
  email
  phone
  other providers
```

Each connection must show status, authority type, verification time, last observation, revocation affordance where applicable, and the consequence of changing or removing it.

9.4 Send

Primary input accepts:

- Prism ID;
- Telegram username;
- X username;
- future email or phone;
- native address as an advanced fallback.

The flow must display:

```
recipient entered
-> provider and subject resolved
-> Prism identity resolved
-> purpose and venue selected
-> privacy route selected
-> fee and public/private metadata explained
-> user authorization requested
-> operation tracked
-> receipt reconciled
```

If the recipient is unregistered, the user sees a PrismClaim option instead of being silently forced back to a raw address.

9.5 Receive

Receive answers:

- How can someone address me?
- Where will the payment land?
- Is the route public or private?
- What information must I disclose?

Possible receive identifiers:

```
prism:P7F21
@alice on Telegram
@alice_xyz on X
QR code
claim link
```

A shareable identifier must not be presented as equivalent to a public wallet address.

9.6 Activity and receipts

Activity is a reconciled, provenance-bearing view of operations. Each row must distinguish:

```
intent
submitted
confirmed
indexed
reconciled
completed
```

A receipt must include, where relevant:

- operation ID;
- purpose;
- venue;
- transaction or provider identifier;
- observed status;
- block or finality metadata;
- public and private fields;
- evidence links;
- next user action if incomplete.

9.7 Profile

Profile is the human-facing view of the persistent identity. It may include:

- Prism ID;
- verified principals;
- approved public presentation metadata;
- receive options;
- active and revoked connections;
- disclosure controls.

Profile must not become a public social graph or expose private bindings by default.

10. Functional requirements

Priority meanings:

- **P0:** required for the relevant release or the flow is invalid without it.
- **P1:** required for a credible product path but can follow the initial Core v1 closure.
- **P2:** valuable extension that must not block the current release.

10.1 Identity and authority requirements

ID	Priority	Requirement	Acceptance condition
PRISM-ID-001	P0	Create a persistent Prism ID on Starknet	A successful creation returns a stable Prism ID and a receipt-backed identity record
PRISM-ID-002	P0	Read a Prism identity independently of bindings	Identity remains readable after binding creation, revocation, and account rotation
PRISM-ID-003	P0	Keep identity, controller, and execution identity distinct	UI, API, and contract schemas use separate fields and do not equate them
PRISM-ID-004	P0	Require controller authority for protected identity mutation	Wrong controller attempts fail with stable authorization errors
PRISM-ID-005	P0	Prove control before accepting an external account	Bind requires a valid domain-, nonce-, expiry-, account-, and Prism-ID-bound proof
PRISM-ID-006	P0	Support the accepted Base proof ladder	EOA, EIP-1271, and ERC-6492 paths are tested and produce distinct failure states
PRISM-ID-007	P0	Make binding canonical only at the Starknet transition	A verified backend proof without the accepted Starknet state transition is not active
PRISM-ID-008	P0	Support binding revocation	Revoked bindings cannot resolve as active and the parent Prism ID persists
PRISM-ID-009	P0	Prevent proof replay	Consumed proof digests cannot be reused across identities or bindings
PRISM-ID-010	P1	Support rebind after revocation	A fresh proof can create a new binding version without rewriting historical evidence

10.2 Resolution and external principal requirements

ID	Priority	Requirement	Acceptance condition
PRISM-RES-001	P0	Resolve by purpose and venue	The resolver returns a capability appropriate to the requested purpose or a typed failure
PRISM-RES-002	P0	Never return revoked state as active	Resolver tests cover unknown, unbound, active, and revoked states
PRISM-RES-003	P0	Use stable external subject IDs	Handle changes update presentation metadata without transferring the binding
PRISM-RES-004	P0	Verify social principals through the provider	An unverified or stale social handle cannot authorize payment or communication
PRISM-RES-005	P1	Avoid public identity graph enumeration	Public surfaces expose the minimum approved metadata and do not dump all bindings

ID	Priority	Requirement	Acceptance condition
PRISM-RES-006	P1	Require re-verification for sensitive principal changes	Account transfer, revocation, or rebind cannot rely on a stale cached handle
PRISM-RES-007	P1	Normalize confusable and ambiguous identifiers	Unicode, lookalike, reassigned, and duplicate identifiers fail safely or require confirmation

10.3 STRK20 private financial requirements

ID	Priority	Requirement	Acceptance condition
PRISM-STRK-001	P0 for STRK20 release	Detect wallet capability by supported API/spec queries	Capability detection does not invoke private-balance reads or handle viewing keys
PRISM-STRK-002	P0	Verify network before private action	The UI blocks private readiness on wrong, unknown, or mismatched network
PRISM-STRK-003	P0	Keep viewing keys in the wallet boundary	No dapp, backend, log, API payload, or evidence file stores a viewing key
PRISM-STRK-004	P0	Represent the two-step shield honestly	Approval and pool deposit are shown as one goal with truthful substeps
PRISM-STRK-005	P0	Require accepted receipt before confirmation	Submitted hash alone cannot advance the flow to confirmed or available
PRISM-STRK-006	P0	Represent note maturity	A fresh note remains maturing until the configured block threshold is observed
PRISM-STRK-007	P0	Require intentional private-balance consent	Private balance is read only after explicit consent and remains provider-owned data
PRISM-STRK-008	P0	Execute a private transfer through a supported route	A real wallet/provider call returns an accepted receipt and the appropriate pool event
PRISM-STRK-009	P0	Read current pool fee	Fees are observed from the pool and included in MAX/minimum useful amount calculations
PRISM-STRK-010	P0	Distinguish screening rejection and screening unavailability	The user sees different recovery guidance for each condition
PRISM-STRK-011	P0	Avoid relayer sender attribution	User activity is correlated from protocol events/state, not transaction sender
PRISM-STRK-012	P1	Show private and public fields separately	The UI names what is hidden, what remains visible, and what may be correlatable
PRISM-STRK-013	P1	Support a Prism-owned meaningful private action	The action is product-native, pool-integrated, reviewed, and capable of independent evidence

10.4 Payments, claims, and relationships

ID	Priority	Requirement	Acceptance condition
PRISM-PAY-001	P0	Let a sender address a recipient by Prism ID or verified principal	Primary flow does not require a raw wallet address
PRISM-PAY-002	P0	Resolve the recipient before authorization	The sender sees provider, verification, purpose, venue, and privacy route

ID	Priority	Requirement	Acceptance condition
PRISM-PAY-003	P1	Support private payment to a registered recipient	A supported private route produces a receipt linked to the resolved recipient capability
PRISM-CLAIM-001	P1	Create a claim for an unregistered recipient	Claim stores a commitment, asset, amount, expiry, policy, and explicit status
PRISM-CLAIM-002	P1	Let the recipient onboard and claim	Recipient proves control of the external principal and receives the claim through the accepted route
PRISM-CLAIM-003	P1	Provide expiry and refund	Expired claims do not strand value and the sender can recover through the defined path
PRISM-CHAN-001	P1	Create a private channel between Prism identities	Participants accept the channel and separate communication key commitments are established
PRISM-CHAN-002	P1	Carry a payment memo, claim invitation, or receipt reference	Ciphertext or commitment is linked to a payment object without placing plaintext onchain
PRISM-CHAN-003	P1	Revoke or archive a channel	Revoked or archived channels cannot silently grant payment or execution authority
PRISM-CHAN-004	P1	Keep communication keys separate	Channel implementation never extracts STRK20 viewing keys or execution keys

10.5 Operations, API, SDK, and agent requirements

ID	Priority	Requirement	Acceptance condition
PRISM-OPS-001	P0	Create an operation row before broadcast	Every write has an operation ID and initial state before an external submission
PRISM-OPS-002	P0	Preserve idempotency	Reusing the same idempotency key with the same fingerprint is benign; a different fingerprint fails
PRISM-OPS-003	P0	Use expected-version guards	Concurrent updates fail closed on stale versions and do not overwrite newer state
PRISM-OPS-004	P0	Reconcile submitted operations	A timeout after submission becomes ambiguous or requires attention, not an automatic retry
PRISM-OPS-005	P0	Preserve stable domain errors	Errors identify category, retryability, user action, and HTTP status hint
PRISM-OPS-006	P0	Keep REST, SDK, and MCP semantics identical	Each surface maps to the same application service and operation states
PRISM-OPS-007	P1	Publish versioned domain-first API	Default API vocabulary does not expose raw felts, calldata, or wallet internals
PRISM-OPS-008	P1	Provide typed SDK methods	Identity, bindings, resolution, payments, claims, channels, operations, receipts, and privacy use typed methods
PRISM-OPS-009	P1	Expose safe MCP tools	MCP may read, create intents, request pauses, and request approval, but cannot sign, bypass policy, read viewing keys, or mark completion
PRISM-OPS-010	P0	Persist canonical facts before advancing checkpoints	Restart and replay do not lose accepted events or advance watermarks ahead of stored facts

10.6 Security, privacy, and release requirements

ID	Priority	Requirement	Acceptance condition
PRISM-SEC-001	P0	Never commit or request secrets	No private keys, seed phrases, RPC secrets, OAuth secrets, viewing keys, or prover material appear in code, prompts, logs, or evidence
PRISM-SEC-002	P0	Separate authentication from authority	Product session, Starknet controller, Base signer, STRK20 wallet, and social verification remain separate fields and checks
PRISM-SEC-003	P0	Enforce venue-native authorization	Base actions use Base-valid authority; Starknet actions use Starknet-valid authority
PRISM-SEC-004	P0	Keep privacy claims observer-specific	Every privacy claim names visible and hidden data plus correlation limitations
PRISM-SEC-005	P0	Keep ordinary Base privacy honest	Base activity is public unless a real Base privacy mechanism is integrated
PRISM-SEC-006	P0	Ensure failure recovery is legible	Reverted, expired, rejected, unavailable, pending, and requires-attention states include recovery guidance
PRISM-REL-001	P0	Maintain a release track and exact candidate	Branch, commit, source set, network, and evidence packet are frozen before consequential execution
PRISM-REL-002	P0	Require independent reads for live state	Receipt, address, class or bytecode identity, block, event, and state are read through an independent path
PRISM-REL-003	P0 for sprint	Validate every selected transaction	Each hash exists on SN_MAIN, succeeded, contains a STRK20 pool event, and satisfies the own-contract rule when contracts are declared
PRISM-REL-004	P0 for sprint	Populate <code>strk20.json</code> only from observed facts	No fixture, placeholder, inferred, testnet, or self-reported value is included
PRISM-REL-005	P0 for public release	Verify the public demo in a clean browser	HTML, assets, hydration, interaction, responsive behavior, reduced motion, and errors are checked
PRISM-REL-006	P0 for public release	Keep demo and video claims bounded	The narrative does not claim mainnet, privacy, or completion beyond the evidence ledger

11. Core workflows

11.1 Identity creation

```
User opens Prism
-> connects or creates a Starknet controller
-> requests Prism ID creation
-> signs through the native Starknet authority
-> PrismIdentityRegistry records identity
-> Prism receives accepted receipt
-> Home shows Prism ID and creation evidence
```

Failure behavior:

- wrong network blocks submission;

- rejected signature returns to authorization state;
- accepted submission remains pending until receipt observation;
- ambiguous delivery disables duplicate retry until reconciliation.

11.2 Base ownership binding

```
User selects Connect Base
-> Prism issues domain-bound challenge
-> Base account signs
-> backend verifies EOA, EIP-1271, or ERC-6492 path
-> user controller authorizes Starknet bind
-> registry consumes proof digest
-> resolver returns active Base destination
```

The backend is a trusted verifier for proof validity. It is not the canonical owner of Prism identity state. Canonical acceptance occurs at the Starknet state transition.

11.3 Revocation and replacement

```
User selects a binding
-> reviews venue, account, purpose, and consequence
-> controller authorizes revoke
-> registry records revocation
-> resolver stops returning the binding as active
-> Prism ID remains readable
-> user may create a fresh binding with a fresh proof
```

11.4 STRK20 private money

```
Connect privacy-enabled wallet
-> observe supportedWalletApi and supportedSpecs
-> confirm SN_MAIN or selected network
-> register viewing key through wallet if required
-> approve token
-> shield through pool
-> wait for accepted receipt
-> show maturing state
-> wait for maturity target
-> request private-balance consent
-> show private balance
-> execute private transfer or meaningful private action
-> observe receipt, pool event, and private-state effect
-> reconcile operation
```

Prism never stores the viewing key. The user sees the privacy boundary rather than a generic private badge.

11.5 Human-addressed private payment

```
Sender enters @alice
-> provider resolves stable subject
-> Prism verifies principal freshness
-> Prism resolves Alice's Prism ID
-> Prism checks private receive capability
-> sender reviews privacy and fee information
-> wallet authorizes private action
-> operation reconciles
-> receipt is attached to the relationship context
```

11.6 Unregistered recipient claim

```
Sender enters @bob
-> no active Prism identity found
-> provider principal is verified
-> sender reviews expiry and refund policy
-> sender authorizes claim funding
-> invitation is sent through an approved channel
-> Bob onboards and proves control
-> Bob creates or links a Prism ID
-> Bob claims
```

11.7 Private relationship channel

```
A selects B
-> channel proposal created
-> B accepts
-> communication key commitments established
-> encrypted payment memo or receipt reference sent
-> authorized participant decrypts
-> channel can be archived or revoked
```

A channel does not automatically authorize payment, execution, delegation, or identity mutation.

11.8 Governed agent or application action

```
Agent or application creates intent
-> Prism validates identity, purpose, policy, and authority
-> Pause created when required
-> verification and approval requested
-> operation released only within scope
-> native adapter executes
-> receipt reconciled
-> Activity exposes final state
```

An agent may request and prepare. It may not silently bypass Pause, sign with user keys, read viewing keys, expand policy scope, or mark settlement complete.

12. State models

12.1 Binding lifecycle

```
unbound
-> challenge_issued
-> proof_submitted
-> proof_verified
-> bind_pending
-> active
-> revoked
-> rebind_pending
-> active
```

Failure states:

```
expired
replay_rejected
wrong_signer
wrong_controller
provider_failure
stale_version
requires_attention
```

12.2 General operation lifecycle

```
created
-> awaiting_authorization
-> ready
-> submitted
-> processing
-> confirming
-> confirmed
-> indexed
-> reconciled
-> completed
```

Terminal or recovery states:

```
failed_retryable
failed_terminal
reverted
expired
cancelled
requires_attention
```

12.3 STRK20 lifecycle

```
capability_unknown
-> mismatch | registration_required | approval_pending
-> shielding
-> confirmed
-> maturing
-> privately_available
-> proving
-> transfer_pending
-> transfer_confirmed
```

Additional states:

```
rejected
dependency_failure
```

The UI must never treat `submitted`, `processing`, `confirming`, or `maturing` as final completion.

12.4 Claim lifecycle

```
created
-> awaiting_sender_authorization
-> funded
-> recipient_unregistered
-> invitation_sent
-> recipient_verified
-> claimable
-> claiming
-> claimed
```

Refund branch:

```
funded
-> expired
-> refund_available
-> refunding
-> refunded
```

12.5 Channel lifecycle

```
proposed
-> accepted
-> active
-> muted
-> archived
-> revoked
```

Inbound request branch:

```
request_received
-> accepted | rejected
```

13. Data model requirements

13.1 Core objects

```
PrismIdentity  
Controller  
ExecutionIdentity  
ExternalPrincipal  
Binding  
Resolution  
ResolutionPolicy  
Portfolio  
PrivateBalance  
PrismClaim  
PrismChannel  
ChannelMessage  
Operation  
Receipt  
EvidenceRecord
```

13.2 Deferred objects

```
Guardian  
Successor  
Delegation  
AgentAuthority  
RecoveryPolicy  
InheritancePolicy  
SelectiveDisclosureProof  
PrivateBindingProof
```

Deferred objects must not be simulated as if they are current capabilities.

13.3 Sensitive data classes

The following must remain outside normal application storage unless a future accepted design explicitly requires a protected representation:

- STRK20 viewing keys;
- raw private proofs or proving material;
- Starknet or Base private keys;
- seed phrases;
- OAuth client secrets and provider tokens;
- plaintext private communication content;
- unnecessary social linkage data;
- unredacted provider diagnostics containing sensitive values.

13.4 Evidence record

Every consequential live or public claim should map to an evidence record containing:

```
evidence_id:
claim:
environment:
network:
chain_id:
timestamp:
commit_sha:
artifact_hash:
operation_id:
transaction_hash:
block:
execution_status:
finality_status:
contract_or_provider:
events:
independent_reads:
privacy_claim_supported:
privacy_claim_not_made:
reproduction_command:
limitations:
status:
```

An evidence record does not automatically promote maturity. The promotion must match the evidence actually observed.

14. API and SDK requirements

14.1 Domain-first REST surface

The public API should expose stable product concepts rather than raw chain mechanics.

```
POST /v1/identity
GET /v1/identity/:prism_id

POST /v1/identity/:prism_id/bindings
POST /v1/identity/:prism_id/bindings/:id/revoke
GET /v1/resolve/:identifier

POST /v1/principals/verify
GET /v1/principals/:platform/:handle

GET /v1/portfolio/:prism_id

POST /v1/payments/prepare
POST /v1/payments/submit

POST /v1/claims
POST /v1/claims/:id/claim
POST /v1/claims/:id/refund

POST /v1/channels
GET /v1/channels/:id
POST /v1/channels/:id/messages

GET /v1/operations/:id
GET /v1/receipts/:id
```

The repository's current versioned OpenAPI contract remains the implementation-level source for exact routes, headers, schemas, and error codes. This PRD defines product requirements, not a replacement API specification.

14.2 SDK vocabulary

```
prism.identities.create()
prism.identities.get()
prism.identities.resolve()

prism.bindings.create()
prism.bindings.revoke()

prism.principals.verifyTelegram()
prism.principals.verifyX()

prism.payments.sendPrivate()
prism.claims.create()
prism.claims.claim()
prism.claims.refund()

prism.channels.open()
prism.channels.send()
prism.channels.archive()

prism.private.capability()
prism.private.balance()
prism.private.transfer()

prism.operations.get()
prism.receipts.get()
```

Low-level escape hatches may exist for advanced developers, but the primary vocabulary must not require users to reason in felts, calldata, class hashes, raw invoke versions, note discovery, or wallet-standard internals.

14.3 Required headers and controls

The application API must support:

- explicit API version;
- request ID;
- correlation ID;
- idempotency key;
- expected-version or conditional update;
- canonical watermark where relevant;
- authenticated session separate from execution authority.

15. Privacy and threat requirements

15.1 Privacy categories

Prism must treat these as separate properties:

```
identity privacy
transaction privacy
relationship privacy
amount privacy
timing privacy
communication-content privacy
communication-metadata privacy
cross-chain linkage privacy
social-linkage privacy
```

No feature inherits a privacy claim from another feature automatically.

15.2 STRK20 privacy truth table

Route	Private or protected	Public or potentially visible
Registration	Viewing-key material remains wallet-owned	Registration event and account relationship required by the protocol
Shield/deposit	Resulting private note state	Depositor, token, amount, timing, and deposit transaction
Note-to-note transfer	Sender, recipient, amount, token relationship, spent-note relation in the supported privacy model	Required proof, nullifier, encrypted note artifacts, timing metadata
Open note	Ownership relationship may be hidden	Amount may be visible
Private DeFi action	Direct user linkage may be hidden behind the protocol route	Action, target, amount, timing, and open-note effects may be visible
Withdrawal	Link to original private history may be reduced	Destination, amount, and timing
Ordinary Base action	No private property by default	Native Base transaction data

The UI must express these distinctions at the point of action, not only in documentation.

15.3 Social-principal threats

Threats include:

- username reassignment;
- compromised social account;
- stale cached profile;
- OAuth token theft;
- binding replay;
- fake provider adapter;
- lookalike or Unicode-confusable username;
- unauthorized rebind;
- public social graph enumeration.

Mitigations include stable subject IDs, fresh verification, domain separation, nonce and expiry, canonical normalization, revocation, rate limits, and sensitive-change re-verification.

15.4 Operation threats

Threats include:

- submission succeeded while receipt polling timed out;
- provider disagreement;
- stale cache;
- chain reorganization or finality uncertainty;
- duplicate retry after ambiguous delivery;
- operation state overwritten by a concurrent worker;
- UI success shown before reconciliation;
- relay sender incorrectly treated as user attribution.

The operation model must preserve ambiguity and require reconciliation before another broadcast.

16. Product quality requirements

16.1 Comprehension

A cold user should be able to answer within the first interaction:

- What is Prism?
- What is my Prism ID?
- Which accounts are connected?
- Which identity can someone use to reach me?
- What can I do now?
- What is private?
- What is pending?

16.2 Truthful degraded states

Every unavailable dependency must produce a useful state:

```
wallet not connected
wallet does not support STRK20
wrong network
registration required
approval pending
screening rejected
screening unavailable
maturing
provider unavailable
receipt pending
provider disagreement
requires attention
```

No blank panel, optimistic success badge, fake balance, or silent fallback is acceptable.

16.3 Visual language

The interface should be:


```
quiet  
premium  
institutional-grade  
human  
globally comprehensible
```

Avoid:

```
chain-logo overload  
neon protocol theatre  
raw infrastructure jargon  
fake transactions  
fake balances  
chat-app imitation  
AI-generated visual clutter
```

Visual hierarchy:

```
identity  
-> total state  
-> private state  
-> actions  
-> people and accounts  
-> activity  
-> evidence
```

16.4 Runtime acceptance

A route is not accepted because it returns HTTP 200. Runtime verification must cover:

- exact branch and commit;
 - server command and port;
 - public URL;
 - HTML and asset status;
 - hydration;
 - accessible interaction;
 - immediate state change after primary action;
 - motion and settled state;
 - desktop and mobile geometry;
 - reduced-motion behavior;
 - browser and server errors;
 - screenshot or reproducible evidence.
-

17. Evidence and release requirements

17.1 Core v1 release gate

Core v1 may be called locally implemented when the relevant code and tests pass. It may not be called mainnet-ready until the accepted release packet contains the required network, deployment, authentication, funding, rollback, receipt, independent-read, and security evidence.

Core v1 scope currently includes:

- Registry V2 as the canonical immutable identity registry;
- persistent Prism identity;
- Base ownership proof and controller binding;
- resolve and revoke lifecycle;
- pause and governance boundary;
- durable operations, events, checkpoints, and reconciliation;
- truthful Home and operation surfaces.

17.2 STRK20 submission gate

The STRK20 submission path requires a public open-source repository with a license, a public live demo, a three-minute demo video, and at least three successful Starknet mainnet transactions touching the live pool, recorded in the root `strk20.json`.^{[3][4]}

For each final hash, Prism must independently confirm:

```
hash exists on SN_MAIN
execution succeeded
STRK20 pool event exists
transaction is distinct
```

If `strk20.json.contracts` is non-empty, every final hash must also involve at least one declared Prism contract under the current validator logic.^[4]

The identity registry alone is not enough to satisfy that own-contract condition because identity operations do not inherently touch the STRK20 pool. A Prism-owned helper or claim path must be meaningful, reviewed, pool-integrated, and independently evidenced before it is declared.

17.3 Evidence promotion

```
unverified
-> simulated pass
-> local pass
-> fork pass
-> testnet pass
-> live pass
-> public evaluator pass
```

A simulated pass cannot satisfy a live or public onchain requirement. A production-looking demo cannot promote an unverified protocol state.

17.4 `strk20.json` policy

The initial file remains:

```
{
  "transactions": [],
  "contracts": [],
  "demo_video": "",
  "demo_url": ""
}
```

Values may be added only when independently observed and separately authorized. Testnet hashes, local fixtures, placeholder URLs, inferred contract addresses, and self-reported transaction results are prohibited.

18. Release roadmap

Phase 0: Product and truth freeze

Objective: preserve the concept and define evidence boundaries.

Deliverables:

- Product Truth and canonical product sentence;
- protected decision reconciliation;
- domain model;
- authority map;
- privacy truth table;
- evidence ledger and release packet shape;
- explicit current, proposed, deferred, and unknown states.

Exit condition: no unresolved contradiction is silently represented as implementation truth.

Phase 1: Core identity substrate

Objective: establish persistent identity and native authority.

Deliverables:

- Registry V2 implementation;
- create and read;
- controller semantics;
- exact proof digest representation;
- event and replay rules;
- local adversarial tests.

Exit condition: local tests pass and the remaining live deployment gates are visible.

Phase 2: Base binding and resolution

Objective: prove that one Prism identity can coordinate a native Base account without conflating the account with identity.

Deliverables:

- challenge issue and verification;

- EOA, EIP-1271, and ERC-6492 ladder;
- controller-authorized Starknet bind;
- resolve;
- revoke;
- independent testnet evidence.

Exit condition: decisive create -> prove -> bind -> resolve -> revoke sequence passes with receipts and independent reads.

Phase 3: Protocol surfaces

Objective: expose one authority path for application and agent clients.

Deliverables:

- versioned REST API;
- SDK;
- optional MCP adapter when agent-mediated operations are part of the promise;
- durable OperationStore;
- Pause and approval semantics;
- reconciliation and receipt surfaces.

Exit condition: REST, SDK, and MCP produce identical operation semantics and cannot bypass authority or policy.

Phase 4: STRK20 private financial surface

Objective: make private state and private action understandable and real.

Deliverables:

- wallet capability detection;
- explicit network state;
- two-step shield UX;
- maturity state;
- consented private balance;
- private transfer;
- current fee observation;
- screening error distinction;
- receipt and privacy limitation record.

Exit condition: supported wallet/provider performs a real pool action and Prism independently reconciles the result.

Phase 5: Prism-owned private application action

Objective: connect private financial infrastructure to Prism's own product idea.

Preferred candidate: PrismClaim, because it unifies a human identifier, an unregistered recipient, private value, onboarding, persistent identity, and claim settlement.

Alternative candidate: private capital allocation through a reviewed Vesu helper if that route has lower risk and stronger product evidence.

Exit condition: selected mechanism passes interface review, contract tests, pool integration tests, privacy review, deployment verification, and independent live evidence.

Phase 6: Social identity and relationship surfaces

Objective: let people address one another through verified human identities.

Deliverables:

- Telegram principal first;
- stable subject ID and handle rotation;
- send and receive by principal;
- PrismClaim invitation;
- minimal PrismChannel payment memo or receipt reference.

Exit condition: one end-to-end relationship flow passes without public graph leakage or authority confusion.

Phase 7: Unified Home and public evaluator hardening

Objective: make the system legible to a cold user and skeptical evaluator.

Deliverables:

- Home;
- Connections;
- Send;
- Receive;
- Activity and receipts;
- privacy labels;
- degraded states;
- public demo;
- video;
- clean browser verification;
- final claim audit.

Exit condition: product and auditor views describe the same observed system.

Phase 8: Continuity and advanced capabilities

Objective: expand the persistent identity into recovery, delegation, and long-term continuity.

Candidates:

- guardians;
- successors;
- recovery policies;
- delegated agents;
- bounded authorities;
- selective disclosure proofs;
- private binding proofs;
- device synchronization.

These capabilities are not allowed to block Core v1 or the first credible STRK20 route.

19. Scope boundaries

In current Core v1 scope

- Starknet-rooted persistent identity;
- Registry V2;
- controller authority;
- Base ownership proof and binding;
- resolution and revocation;
- Pause and governed operations;
- durable backend operation handling;
- truthful Home and operation states.

In the larger Prism product direction

- STRK20 private balances and private transfers;
- private human-addressed payments;
- PrismClaim;
- verified social principals;
- PrismChannel;
- relationship-scoped payment and receipt context;
- native multi-chain execution coordination;
- future continuity and delegation.

Explicitly deferred

- private Base execution;
- Solana execution;
- universal bridge;
- solver network;
- generalized MPC signer;
- full messaging client;
- voice or video communication;
- all social providers;
- all DeFi protocols;
- shadow-account dependency in the normal Wallet API MVP;
- inheritance and guardianship implementation;
- generalized agent wallet authority;
- custom PrismZK for every action;
- public universal balance abstraction without source provenance.

Deferred does not mean rejected. Each deferred capability requires its own Product Truth, system design, security decision, implementation, and evidence gate.

20. Success metrics

These are proposed product targets, not observed results.

User comprehension

- A cold user can state what Prism is without reading the architecture documents.
- A cold user can locate the Prism ID, connected accounts, private state, and primary action quickly.
- A user can distinguish public, private, pending, and completed states from the UI alone.

Identity and authority integrity

- Zero accepted bindings without the required proof and controller authorization.
- Zero active resolutions returned for revoked bindings.
- Zero social rebindings caused only by a changed display handle.
- Zero protocol authority inferred from product authentication alone.

Privacy integrity

- Zero application paths that request or persist STRK20 viewing keys.
- Zero plaintext private relationship content placed onchain by default.
- Every private claim names its visible fields and limitations.
- No public social graph is emitted as a side effect of resolution.

Operational integrity

- Every write has an operation record before external submission.
- Every ambiguous submission is reconciled before retry.
- Every completed state is receipt-backed and independently reconciled.
- REST, SDK, and MCP produce the same operation and policy semantics.

Evidence integrity

- No `strk20.json` entry without independent verification.
- No mainnet claim based only on local tests, fixtures, dry-runs, or worker reports.
- Every public demo claim maps to a product state and evidence record.

21. Risks and mitigations

Risk	Severity	Mitigation
Wallet does not support the required STRK20 methods	Critical	Probe capability immediately and stop before deepening the route
Mainnet prover or discovery dependency is unavailable	Critical	Prefer supported Wallet API; mark downstream route blocked rather than inventing endpoints
Custom helper expands beyond reviewable scope	Critical	Use a small meaningful PrismClaim or approved helper spike with an early kill time

Risk	Severity	Mitigation
Final hashes touch the pool but not declared Prism contracts	Critical	Decide contract declaration before final hash collection and run hub-equivalent validation
Prism collapses into an address book	High	Preserve identity, authority, purpose, revocation, and continuity as first-class concepts
Prism becomes a wallet clone	High	Keep persistent identity, social principals, claims, relationships, and native execution coordination central
UI overstates privacy	High	Use observer-specific labels and privacy claim review at every release
Public demo uses stale or broken assets	High	Fresh production server, all-asset check, browser hydration, interaction, responsive, and reduced-motion verification
Backend cache becomes de facto authority	High	Canonical-source map, watermarks, independent reads, and explicit derived labels
Social username reassignment redirects authority	High	Stable subject IDs, re-verification, revocation, and sensitive-change policy
Ambiguous provider delivery causes duplicate broadcast	High	Durable operation ID, submission fence, reconciliation before retry
Shadow accounts or broad DeFi scope consumes the release window	Medium	Keep them deferred unless they directly strengthen the selected proof
Full channel product displaces the identity and payment core	Medium	Implement only relationship, memo, receipt, and key-boundary slices first

22. Open decisions

The following items remain open or require re-verification before the affected capability becomes canonical:

1. Which supported Wallet API implementation is available for the live target network?
2. Is a hosted discovery service required for the selected Wallet API version?
3. Which Prism-owned private application action is selected for final evidence: PrismClaim or another accepted helper route?
4. Will final STRK20 evidence declare project contracts, and if so, which exact contracts and versions?
5. Which social provider is first in production, and what provider subject and re-verification guarantees are available?
6. What storage layer is selected for PrismChannel ciphertext, commitments, discovery, and device recovery?
7. Which release claims include PrismChannel, PrismClaim, or agent-mediated operations as mandatory rather than future capabilities?
8. What is the accepted mainnet owner decision, signer scope, funding reserve, rollback policy, and stop condition?
9. What exact date and time does the organizer's extended deadline use? The current working assumption is September 7, 2026 from the owner-provided announcement. The official sprint surfaces must be refreshed before final submission freeze.
10. Which production authentication issuer, managed database, telemetry, and alerting configuration is accepted for a mainnet release?

An open decision is not permission to infer a default. Where the decision changes authority, privacy, contract, or release behavior, the product must stop at the decision boundary.

23. Traceability map

Product requirement area	Canonical project source
Product definition and Home	docs/PRISM_DOCUMENTATION_V0_3.md
Core v1 scope	projects/prism/CORE_V1_CLOSEOUT_SCOPE.md
Product and system decisions	projects/prism/DECISIONS.md
System authority and architecture	projects/prism/system/SYSTEM_CANONICAL.md
API, SDK, MCP, and channel phases	projects/prism/agent-packets/PRISM_PROTOCOL_SURFACE_PHASE_PLAN.md
STRK20 product context and privacy boundaries	docs/STRK20_CONTEXT.md
Live and submission evidence	profiles/STARKNET_MAINNET_EVIDENCE_PROFILE.md
Product and live-build audit	projects/prism/AUDIT.md
Mainnet preparation boundary	projects/prism/MAINNET_PREPARATION_HANDOFF.md
Mainnet release packet	ops/release/mainnet-release-packet.template.json
API contract	docs/api/openapi.yaml
Submission artifact	strk20.json

Authority precedence

accepted Product Truth

-> accepted system decisions

-> verified ecosystem constraints

-> implementation

-> runtime observation

-> public claim

A lower layer may implement or project an upper-layer decision. It may not silently redefine it.

24. Definition of done for the full Prism concept

The full Prism concept is not complete because every future feature exists. It is complete as a product system when the current release is bounded, the larger direction is visible, and each promised capability has a truthful path from user intent to authority, execution, reconciliation, and evidence.

For any capability to be called complete, all applicable items must be true:

```
product problem and user job are explicit
+ authority and canonical source are explicit
+ privacy claim and visible data are explicit
+ UI state exists
+ API/application path exists
+ failure and recovery states exist
+ local tests pass
+ external interface is verified
+ live evidence exists when claimed
+ independent readback exists for consequential state
+ public wording matches the evidence ledger
```

For the current Core v1 and STRK20 sprint path, the immediate order remains:

```
G0 real STRK20 reachability
-> wallet capability vertical slice
-> Core identity and binding evidence
-> private balance and transfer
-> Prism-owned private application decision
-> qualifying mainnet receipts
-> Home / Activity / Connections integration
-> public demo and video
-> final evidence and release freeze
```

Prism should build only what can become evidence, and every piece of evidence must remain attached to the exact product claim it proves.

Sources

- [3] <https://strk20.starknet.io/hackathon> — STRK20 Private Sprint official hub
- [4] <https://github.com/starkience/strk20-hackathon> — STRK20 Private Sprint official repository
- [10] <https://raw.githubusercontent.com/starkience/strk20-hackathon/main/docs/MAINNET-DAY-0.md> — STRK20 Day 0 mainnet guide
- [11] <https://strk20-by-example.org/what-is-strk20> — STRK20 by Example
- [12] <https://github.com/starkware-libs/starknet-privacy> — Starknet Privacy repository